

1. [Multiple Choices, 40 points] For each of the following questions, select only ONE option that best answers the question.

(1) [2 points] Which of the following is NOT among the supported parameters in REST API of Firebase realtime database?

- a) orderBy
- b) limitToFirst
- c) startAt
- d) groupBy

(2) [2 points] Which of the following data formats is most structured?

- a) JSON
- b) XML
- c) Relation
- d) Text

(3) [2 points] Which of the following MongoDB operators is used to join two MongoDB collections?

- a) \$group
- b) \$match
- c) \$lookup
- d) \$unwind

(4) [2 points] Which of the following is **most** likely to be the page size of SSD?

- a) 4B
- b) 4KB
- c) 4MB
- d) 4GB

(5) [2 points] Which operations in SSD are conducted page by page?

- a) Read only
- b) Read and write
- c) Write only
- d) Erase only

(6) [3 points] Which of the following is the correct order of tasks being executed in Hadoop MapReduce?

- a) Map, shuffle, reduce

- b) Map, reduce, shuffle
- c) Shuffle, map, reduce
- d) Reduce, map, shuffle

(7) [3 points] Consider joining two relations R and S with the amount of memory M (pages). Sizes (i.e., the number of blocks) of R and S are denoted as $B(R)$ and $B(S)$ respectively. Suppose both $B(R)$ and $B(S)$ is larger than M. Which of the following statements about choices of join algorithms is **correct**?

- a) Nested-loop join algorithm always has higher cost than sort-merge join algorithm.
- b) Nested-loop join algorithm always has higher cost than partitioned-hash join algorithm.
- c) Sort-merge join algorithm always has the same cost as the partitioned hash join algorithm.
- d) If $B(R) < B(S)$, then placing R in the outer loop of the nested-loop join will incur a lower cost.

(8) [3 points] Which of the following transformations in Spark involves shuffling?

- a) map
- b) filter
- c) flatMap
- d) reduceByKey

(9) [3 points] Suppose there are some **empty** *author* elements in a book XML document. Which of the following XPath expressions correctly finds such elements?

- a) `//author[node()]`
- b) `//author[*]`
- c) `//author[not(node())]`
- d) `//author[not(text())]`

(10) [3 points] Consider an XML element about book:

```
<book isbn=1234>  
  <author>John</author>  
  <author>123</author>
```

```
<isbn>5678</isbn>
<title>Database</title>
</book>
```

What syntax error does the above book element have?

- a) It has isbn as both attribute and sub-element.
- b) The value of the isbn attribute (1234) needs to be quoted.
- c) It can not have multiple author subelements.
- d) None of above.

(11) [3 points] Suppose the **actual bandwidth** of reading 100 and 200 **random** blocks of data from a hard drive is B1 and B2 respectively. Which of the following statements about the relationship between B1 and B2 is most accurate?

- a) $B1 < B2$
- b) $B1 > B2$
- c) $B1 = B2$
- d) $B2 = 2*B1$

(12) [3 points] How many HDFS blocks are needed to store 1GB of data?

- a) 2
- b) 4
- c) 8
- d) 10

(13) [3 points] Consider two relations:

Product(id int primary key, title varchar(20));

Sales(id int primary key, qty float, foreign key(id) references Product(id));

Suppose Product has 100 rows and Sales has 1000 rows. How many rows will the following query produce:

Select * from Product natural join Sales;

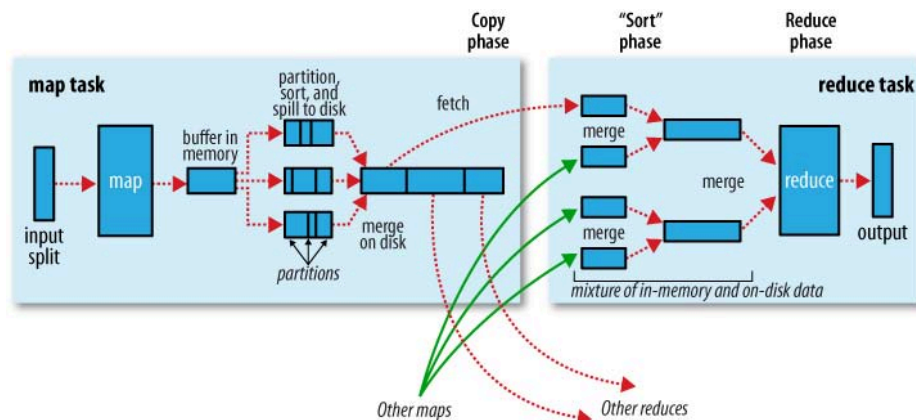
- a) 100
- b) 1000

- c) 900
- d) 1100

(14) [3 points] The following diagram you have seen in class illustrates the shuffling process in Hadoop MapReduce. Recall that flowcharts in boxes marked as map and reduce tasks describe the steps involved in the tasks.

How many map tasks and reduce tasks does the MapReduce job illustrated by the diagram have?

- a) 3 map and 4 reduce tasks
- b) 4 map and 3 reduce tasks
- c) 3 map and 3 reduce tasks
- d) 4 map and 4 reduce tasks



(15) [3 points] Continue from the above question. Which tasks will be merging runs?

- a) Map task only
- b) Reduce task only
- c) Both map and reduce tasks
- d) Neither map nor reduce tasks

2. [Spark DataFrame, 15 points] Consider the **countrylanguage** data you have seen in the homework. Suppose the data is now stored in a CSV file, **countrylanguage.csv**. Every row of the file has values for **CountryCode**, **Language**, **IsOfficial** (either 'T' or 'F'), and **Percentage** (assumed to be an integer).

For example,

CAN,English,T,60
CAN,French,T,23
USA,English,T,86
USA,French,F,5
MEX,Spanish,T,92

Further suppose the **country** data is stored in country.csv and it has only two columns: **Code** and **Name**. For example,

CAN,Canda
USA,United States
MEX,Mexico

Suppose a Spark dataframe called "cl" has been created for you:

```
cl = spark.read.csv('countrylanguage.csv', ...)  
country = spark.read.csv('country.csv', ...)
```

The dataframe also has four columns with data and format corresponding to that in the csv file, that is, the first attributes are strings and the last attribute is an integer.

Also the following import statement has been executed.

```
import pyspark.sql.functions as fc
```

Using the cl (and fc), write a Spark Dataframe script for each of the following SQL queries. Note your script should not return extra information that SQL queries are not asking for, and should return the attributes with names stated in the SQL queries.

(1) [6 points]

```
Select Language, max(Percentage) as max_p  
From CountryLanguage  
Where IsOfficial = 'T'  
Group by Language  
Having count(*) >= 10;
```

```
cl.filter("IsOfficial = 'T']").groupBy('Language').\n  agg(fc.max('Percentage').alias('max_p'), fc.count('*').alias('cnt')).\
```

```
filter('cnt >= 10').select('Language', 'max_p').show()
```

-1 per error; but correct logic

-5 significant misunderstanding but attempts DataFrame operations

(2) [5 points]

```
select country.Name, Language
from   country natural join countrylanguage
where      IsOfficial = 'T';
```

```
country.join(cl, country.Code == cl.CountryCode).filter('IsOfficial = "T"').\
select('Name', 'Language').show()
```

-1 per error; but correct logic

-4 significant misunderstanding but attempts DataFrame operations

(3) [4 points]

```
(select Language
 from countrylanguage
 where IsOfficial = 'F' and CountryCode = "CAN")
except
(select Language
 from countrylanguage
 where IsOfficial = 'F' and CountryCode = "USA");
```

```
can = cl.filter((cl.IsOfficial == 'F') & (cl.CountryCode ==
'CAN')).select('Language')
usa = cl.filter((cl.IsOfficial == 'F') & (cl.CountryCode ==
'USA')).select('Language')

can.subtract(usa).show()
```

-1 per error; but correct logic

+1 significant misunderstanding but attempts DataFrame operations

3. [Query execution, 15 points] Now suppose the country and countrylanguage data are stored in an RDBMS server. Consider the following SQL query:

```
Select *  
From country join countrylanguage  
on country.Code = countrylanguage.CountryCode
```

Suppose we have the following situation:

- $B(\text{country}) = 5,000$, that is, country table occupies 5000 blocks on storage device.
- $B(\text{countrylanguage}) = 10,000$.
- $M = 102$, that is, there are 102 pages in main memory that can be used for the algorithm.

- (1) [7 points] Suppose the server decides to use the (block-based) nested-loop join algorithm, with the country table placed in the outer loop.

- a. [5 points] Describe the algorithm.

For every 100 (or $M-2$) blocks of country (called c_i):
Load every block of countrylanguage (cl_j):
Join them (c_i with cl_j)

-1 per error, but correct logic

+1 Mentions join/nested loop but incorrect details

- b. [2 points] What is the cost (block I/O's) of the algorithm? Show your derivation.

$$5,000 + 5,000/100 * 10,000 = 505,000 \text{ (blocks I/O)}$$

-1 incorrect result, but correct formula

-2 incorrect

- (2) [8 points] Now suppose the server uses the sort-merge join algorithm instead. Assume that 100 blocks of memory is used for sorting and 101 blocks for merging.

- a. [6 points] Describe the steps of executing the algorithm on the above tables. Be sure to indicate the number of runs and the size of runs at each step.

- Sort R => 50 runs, 100 blocks/run
- Sort S => 100 runs, 100 blocks/run
- Further merge 100 runs of S => 1 run
- Merge 50 runs of R with 1 run of S

-1 per error, but correct pipeline
 -2 incorrect merging steps (merge 100 runs into 1 and then merge 50 runs and 1 runs)
 +1 only vague description

- b. [2 points] What is the total cost (block I/O's) of the algorithm on joining the two tables? Show your derivation.

Sorting cost: $2B(R) + 2B(S) + 2B(S)$

Merging: $B(R) + B(S)$

Total: $3B(R) + 5B(S) = 15,000 + 50,000 = 65,000$ (block I/O's)

-1 incorrect result, but correct formula or correct merging cost
 -2 incorrect

4. [Hadoop MapReduce, 15 points] Consider the same countrylanguage.csv data that you have seen in the Spark DataFrame question above. Schema and sample data are reproduced here:

===

Every row of the file has values for **CountryCode**, **Language**, **IsOfficial** (either 'T' or 'F'), and **Percentage**.

For example,

CAN,English,T,60
 CAN,French,T,23
 USA,English,T,86
 USA,French,F,5
 MEX,Spanish,T,92

===

Consider an SQL query:

```
Select Language, max(Percentage)
From CountryLanguage
Where IsOfficial = 'T'
Group by Language
Having count(*) > 1
```

Consider finding the answer to the above SQL query using Hadoop MapReduce.

(1) [5 points] Write a pseudocode for the map function of map task.

```
map(key, value):
    toks = value.split(',')

    language = toks[1]
    percentage = toks[-1]
    isofficial = toks[2]

    if isofficial == 'T':
        output(language, percentage)
```

-1 per error, but overall correct

+1 partial attempt but incorrect

(2) [2 points] State the output from the map task for the above sample data.

```
(English, 60)
(French, 23)
(English, 86)
(Spanish, 92)
```

-1 missing 1 pair or slightly off

(3) [5 points] Write a pseudocode for the reduce function of reduce task.

```
reduce(key, values);
    cnt = 0
    max = 0

    for value in values:
```

```
if value > max:
    max = value
cnt += 1
```

```
if cnt > 1:
    output(key, max)
```

-1 per error but logic is mostly correct

+1 partial attempt but incorrect

(4) [3 points] State the input and output of **each** call to the reduce function executed for the above sample data.

Input: (English, [60, 86])

Output: (English, 86)

Input: (French, [23])

Output: None

Input: (Spanish, [92])

Output: None

-1 per error

-3 incorrect

5. [Spark RDD, 15 points]

Note in this question, "sc" refers to a SparkContext object.

(1) [7 points] Consider the following Spark RDD code:

```
lst = [1, 2, 3, 4, 5, 6] # note lst may take other values
```

```
data = sc.parallelize(lst, 2)
```

(a) [5 points] Write a single RDD aggregate function to compute both sum and count of values in data.

```
data.aggregate((0, 0), lambda U, x: (U[0] + x, U[1] + 1),
```

```
lambda U, V: (U[0] + V[0], U[1] + V[1]))
```

-1 per mirror error but logic is mostly correct

-2 per incorrect lambda function
+1 partial attempts but incorrect

(b) [2 points] Explain how the aggregation function computes both sum and count in parallel.

It first computes local sum and count in each partition (by using the first lambda function); it then adds up the local sums and counts to obtain the final sum and count (using the second lambda function).

-1 partial explanation, only mentioning data is splitted into partitions
-2 no explanation or incorrect explanation

6. [8 points] Consider the same countrylanguage.csv data that you have seen in the Spark DataFrame question above. Schema and sample data are reproduced here:

===

Every row of the file has values for **CountryCode**, **Language**, **IsOfficial** (either 'T' or 'F'), and **Percentage**.

For example,

CAN,English,T,60
CAN,French,T,23
USA,English,T,86
USA,French,F,5
MEX,Spanish,T,92

===

Consider an SQL query:

```
Select Language, count(*)  
From CountryLanguage  
Where IsOfficial = 'T'  
Group by Language
```

Suppose the following codes have been executed for you.

```
lines = sc.textFile('CountryLanguage.csv', 2)
```

Using the lines RDD, write a Spark RDD to find out the answer to the above SQL query.

```
result = lines.map(lambda line: line.split(',')) \  
                .filter(lambda t: t[2] == 'T') \  
                .map(lambda t: (t[1], 1)) \  
                .reduceByKey(lambda a, b: a + b)  
  
result.collect()
```

-1 per error but logic is mostly correct
-2 per wrong lambda functions
+1 partial attempt with incorrect logic